

TPX3Rec Architecture Refactoring

Modernizing the X17 Reconstruction Framework

Martin Vavřík

martin.vavrik@utef.cvut.cz
MFF UK & IEAP CTU PRAGUE

July 22, 2026

Summary generated with the help of Gemini 3.1 Pro.



Original Codebase Limitations:

- Heavy reliance on monolithic files and disconnected primitive C-arrays.
- Tight coupling between I/O, configuration, and physics logic.
- Repetitive event loop boilerplate across different execution steps.

New Architectural Goals:

- **Encapsulation:** Group detector physics into robust structural models.
- **Polymorphism:** Standardize execution via a unified Task interface.
- **Separation of Concerns:** Isolate ROOT I/O state and TEnv parsing.
- **Maintainability:** Prepare for safe memory management and LLVM tooling.



Directory Standardization:

- More typical names directory names for source files (`src/`) from the public API (`include/`).

LLVM Tooling Integration:

- **clangd**: Custom `cmake_wrapper.sh` maps the ROOT build, enabling accurate code-completion and jump-to-definition in VS Code.
- **clang-tidy & clang-format**: Enforces a strict, ROOT-inspired formatting standard and flags memory leaks / legacy C-patterns.
- If `compile_commands.json` are generated for ROOT, you can easily jump inside its source files to function definitions



The Old Approach:

- Relied on a monolithic `TPX3Rec_nTuple` class containing over 100 loose arrays and branch pointers.
- Memory pre-allocated simultaneously regardless of the active task.

The New Core Models:

- **Cluster:** Encapsulates topology, local-to-global transformations, and native bounding box/track logic.
- **Coincidence:** Aggregates clusters to compute 3D vertices, angular correlations, and target projections.

Current Bottleneck & Future Fix: To maintain ROOT serialization compatibility, Coincidence currently copies raw Cluster data into flat vectors. Future iterations will upgrade Coincidence to simply hold a `std::vector<Cluster>` natively.

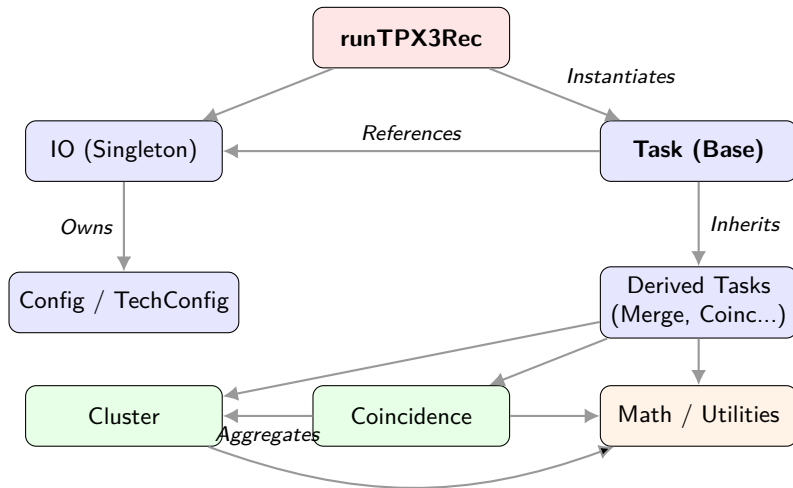


The central steering macro (`runTPX3Rec.cxx`) now delegates execution to isolated, derived tasks.

- **Task (Base Class)**: Owns standard file initialization, standard branching, and finalization. Defines the pure virtual `Execute()` loop.
- **SingleDet & MergeTrees**: Handles standalone calibrations and multi-detector file merging.
- **GetCoincidences**: Core physics loop detecting time-correlated clusters.
- **AlignDetectors**: Global spatial transformation and vertex generation.
- **ProcessSimulation / FilterSimulation**: Monte Carlo pipeline integration.



Dependency Graph



Identified Issues: Math & Geometry

During the architectural audit, several geometric anomalies and legacy C++ traps were identified for future resolution:

- **Type-Safe Math (`std::abs` vs `abs`):** The C++ `abs()` function silently truncates to integers. This likely causes unwanted rounding in functions like the Hough transform (already replaced with `std::abs()`).
- **Geometric Edge Cases:**
 - Linearity can currently evaluate to < 1 for perfectly straight diagonal tracks.
 - `MaxDistance` adds 1 pixel assuming alignment with the X/Y axes, skewing distances by up to $\sqrt{2}$.
 - `BorderHits` assumes hits lie exactly on the `GetRect` bounding box, cascading into `MaxDistance` inaccuracies.
- **Asymmetric Cuts:** Exclusion of border tracks currently only applies to one of the two clusters in `CalculateVertex`.



Architectural Observations & Future Roadmap

Beyond the core models, the codebase review highlighted several areas where the framework's boilerplate can be further optimized:

- **TChain Design Question:** Detector data is currently split across parallel TChain arrays. A more robust ROOT paradigm would be multiplexing the detector ID into a single TTree.
- **Event Loop Overhead:** Functions like `SingleDet` execute redundant `SetBranchAddress` bindings on every iteration (related to converting `std::vectors` to C-style arrays).
- **Code Consolidation:** Duplicate, slightly divergent implementations of `FitTrack`, `Hough2D`, and `GetTargetProjections` need to be deduplicated.
- **Memory & 3D Math Integration:** Swap manual heap allocations (`new[]`) for RAII arrays, and replace primitive variables with native `ROOT::Math` objects.
- **Documentation (Doxygen):** Progressively document the new API headers (`include/`), leveraging AI-assisted generation in VS Code to expedite writing boilerplate.